Ecommerce Recommendation Systems June

DiffRec Fixes · Yelp Dataset

Ananya Tirumala | June 2026

What Changed — Overview

FOUR AREAS OF WORK

Model Changes

- DiffRec: 4 targeted training fixes to address near-zero metrics
- L-DiffRec: LightGCN and Diffrec
- LightGCN: code cleaned up

Infrastructure & Data

- Multi-dataset support: Yelp added alongside Amazon Video Games
- Checkpoint naming: `{dataset}_{model}.pt` to avoid collisions

Frontend

- Dataset toggle (Video Games and Yelp)
- `GameCard` → `ItemCard` (dataset-agnostic)
- Metrics dashboard handles `{dataset}_{model}` checkpoint keys

> The primary motivation was DiffRec's near-zero Recall@10 (0.00725). 4 fixes raised it to 0.0233 without changing the core architecture.

DiffRec Fixes — Timestep Embedding & Normalisation FIX 1 · 2

PROBLEM: RAW SCALAR TIMESTEP + UNNORMALISED INTERACTION VECTORS

Fix 1 — Sinusoidal timestep embedding

Old: pass raw scalar `t` through `Linear(1→64)`; the gradient signal for the time dimension was weak and poorly conditioned.

New: sinusoidal frequency encoding (same family as transformer positional encoding):

$$\text{emb}(t) = \left[\sin\left(\frac{t \cdot \omega_k}{T}\right), \cos\left(\frac{t \cdot \omega_k}{T}\right)\right]_{k=1}^{32}$$

Then projected: `Linear(64→hidden) → SiLU → Linear(hidden→hidden)`

Fix 2 — L2-normalise interaction rows

User interaction vectors are binary and extremely sparse. Without normalisation the vector norm scales with the number of interactions, but Gaussian noise $\mathcal{N}(0, I)$ is unit-scale. This mismatch means diffusion adds proportionally tiny noise to active users, so the denoiser never learns a meaningful reverse mapping.

$$x_0^{\text{norm}} = \frac{x_0}{\|x_0\|_2}$$

> These two fixes address why the denoiser was not learning: the time signal was uninformative and the data scale was incompatible with the noise schedule.

DiffRec Fixes — Loss & Parameterisation FIX 3 · 4

PROBLEM: UNIFORM MSE LOSS

Fix 3 — Density-adaptive weighted loss

On sparse binary data, interaction positions (1s) are vastly outnumbered by zeros. Uniform MSE minimises by predicting near-zero everywhere. New loss:

$$\mathcal{L} = \mathbb{E}\left[w \cdot \|\hat{x}_0 - x_0^{\text{norm}}\|^2\right]$$

$$w_{ij} = 1 + x_{0,ij} \cdot \min\left(\frac{1}{\bar{x}_0}, 5000\right)$$

Interaction positions receive proportionally higher weight, forcing the model to recover the signal at observed entries.

DiffRec Fixes — Inference Alignment & Conditioning FIX 5 · COND

PROBLEM: TRAINING/INFERENCE DISTRIBUTION MISMATCH

Fix 4 — Normalise at inference

Old `recommend()` used raw binary x_0 as context but started from pure Gaussian noise $x_T \sim \mathcal{N}(0, I)$. The model was trained on normalised x_0^{norm}, so inference operated outside the training distribution.

New: normalise x_0 identically at inference, then corrupt to the correct noise level:

$$x_T = \sqrt{\bar{\alpha}_{T-1}}\, x_0^{\text{norm}} + \sqrt{1 - \bar{\alpha}_{T-1}}\, \epsilon$$

This means inference starts from a noisy version of the actual user vector, not pure noise.

L-DiffRec Changes LATENT DIFFUSION

TWO CHANGES TO THE LATENT VARIANT

Instead of running diffusion directly on the full n_items-dimensional interaction vector (which can be 5000+ dimensions), L-DiffRec first compresses it to a small latent vector (latent_dim=64), runs diffusion there, then decodes back to item scores

Encoder: **Linear(n_items→hidden)→GELU→Linear(hidden→64)**

Multi-Dataset Support YELP

AMAZON VIDEO GAMES → VIDEO GAMES + YELP

Data pipeline additions (`training/data_loader.py`)

- `load_meta_jsonl(meta_path)` — reads Amazon `meta_*.jsonl` files; extracts title, brand/developer, categories, image URL per ASIN
- `load_yelp_interactions(review_path)` — reads Yelp `review.json` ; maps `business_id → item_id` , parses ISO date strings to Unix timestamps
- `load_yelp_meta(business_path)` — reads `business.json` ; stores business name, city (as author), category list
- `preprocess_yelp(...)` — entry point for Yelp; delegates to shared `_pipeline`
- `_pipeline(df_raw, item_meta, ...)` — unified filter → split → save flow used by both formats

Metadata is now written to `labels_meta.parquet` and item embeddings are looked up from the meta dict rather than left blank.

Path conventions

Dataset	Processed	Splits
videogames	`data/processed/`	`data/splits/`
yelp	`data/yelp/processed/`	`data/yelp/splits/`

`DATASET_PATHS` dicts in `train_all.py` and `evaluate.py` centralise these.

Checkpoint naming

All checkpoints are now prefixed: `{dataset}_{model}.pt`

e.g. `videogames_lightgcn.pt` , `yelp_neumf.pt`

This lets both datasets share a single `checkpoints/` directory without collision, and the evaluation metrics key (`{dataset}_{model}`) matches the checkpoint name exactly.

Frontend Changes

DATASET TOGGLE · ITEMCARD · METRICS DASHBOARD

Dataset toggle (`App.jsx`)

A pill-style toggle button group lets the user switch between "Amazon Video Games" and "Yelp" without a page reload. Switching clears current results and re-fetches metrics and status for the new dataset. All API calls pass `?dataset=` as a query param.

`GameCard` → **`ItemCard`** (`ResultsGrid.jsx` , new `ItemCard.jsx`)

`GameCard` was hard-coded for book metadata (title, Developer). `ItemCard` is dataset-agnostic: renders title, item ID, developer/city field, and image URL . The component name change reflects the broader scope.

Metrics dashboard (`MetricsDashboard.jsx`)

Metrics JSON now uses `{dataset}_{model}` keys (e.g. `videogames_lightgcn`). The dashboard strips the dataset prefix via `bareModelId(key)` to look up colours and labels:

Bar chart and table legends show the human-readable model label (e.g. "LightGCN") rather than the raw checkpoint key.

A new **Checkpoint** column in the metrics table shows the full checkpoint filename (`videogames_lightgcn.pt`) for traceability.

Serving Changes FLASK API

MULTI-DATASET ROUTING IN `SERVING/APP.PY`

Dynamic DB switching

Old: a single SQLite DB was connected at startup.

New: the DB path is resolved per-request from the `dataset` query param:

```
DB_PATHS = {
    "videogames": "rec_study_videogames.db",
    "yelp":       "rec_study_yelp.db",
}
```

`before_request` closes any open DB connection, re-initialises with the correct path, then reconnects. This lets a single Flask process serve both datasets without restart.

Endpoint changes

All endpoints now accept `?dataset=videogames` (default):

Endpoint	Change
`POST /api/recommend`	body gains `dataset` field
`GET /api/metrics`	reads `metrics_{dataset}.json`
`GET /api/users/sample`	queries dataset-specific DB
`GET /api/status`	queries dataset-specific model registry

`METRICS_PATHS` maps dataset → JSON file path so the `/api/metrics` handler does not need to know the naming convention:

```
METRICS_PATHS = {
    "videogames": "metrics_videogames.json",
    "yelp":       "metrics_yelp.json",
}
```

Updated Results

Model	Dataset	Recall@10	NDCG@10	MRR
LightGCN	Video Games	**0.0625**	**0.0339**	**0.0256**
LightGCN	Yelp	0.0378	0.0215	0.0227
NeuMF	Video Games	0.0328	0.0172	0.0127
NeuMF	Yelp	0.0285	0.0162	0.0178
DiffRec (fixed)	Video Games	**0.0233**	**0.0118**	**0.0085**
L-DiffRec	Video Games	0.0127	0.0067	0.0050
DiffRec (old)	Video Games	0.0073	0.0039	0.0031
DiffRec	Yelp	0.0068	0.0035	0.0032

KEY OBSERVATIONS

DiffRec improved 3× after fixes on Video Games. Recall@10 rose from 0.0073 → 0.0233. The five-fix package (sinusoidal embedding, L2 normalisation, weighted loss, x0-prediction, inference alignment) collectively resolved the near-zero pathology.

DiffRec on Yelp mirrors the pre-fix Video Games baseline. Recall@10 of 0.0068 near-zero, similar to the unfixed Video Games run — suggesting the same pathologies apply on Yelp and the same fixes are needed there.

LightGCN generalises well across domains. Recall@10 drops from 0.0625 (Video Games) to 0.0378 (Yelp), a ~40% reduction likely explained by Yelp's lower density and noisier implicit signals (star ratings vs. purchase events), but graph propagation remains the strongest approach on both datasets.

L-DiffRec still trails DiffRec.